

ЛР2: Реализация микросервисов

Студент: Зеленков Евгений

Группа: БР2.2

Тема: разделение backend-приложения бронирования ресторанов на микросервисы

Цель работы

Цель работы - реализовать локальное разделение монолитного backend из ЛР1 на микросервисы на основе технического дизайна из Д34. В рамках ЛР2 нужно выделить сервисы, разделить владение данными, сохранить внешний API через Gateway и подготовить архитектуру к последующему подключению брокера сообщений и контейнеризации.

Реализованная архитектура

В директории `labs/lab2/app` реализован один npm-проект с несколькими независимыми Node.js entrypoint. Каждый сервис запускается отдельным процессом:

Сервис

Порт

Ответственность

api-gateway

3105

внешняя точка входа /api/v1, проверка JWT, маршрутизация

identity-service

3111

пользователи, роли, регистрация, вход

restaurant-catalog-service

3112

рестораны, кухни, столики

menu-service

3113

меню, блюда, фотографии блюд

reservation-service

3114

бронирования и статусы

review-service

3115

отзывы, фотографии отзывов, модерация

Клиент работает только через api-gateway. Gateway проверяет JWT и передает во внутренние сервисы заголовки X-User-Id, X-User-Role, X-Request-Id и X-Service-Token.

Разделение БД

Для локальной разработки используется один контейнер PostgreSQL, но внутри него созданы пять отдельных баз данных. Это сохраняет принцип database-per-service на уровне схемы владения данными:

Сервис

База данных

identity-service

identity_db

restaurant-catalog-service

restaurant_catalog_db

menu-service

menu_db

reservation-service

reservation_db

review-service

review_db

Прямых внешних ключей между БД разных сервисов нет. Связи между доменами хранятся как внешние UUID: userId, restaurantId, tableId, reservationId.

Межсервисное взаимодействие

Синхронное взаимодействие выполнено через REST:

reservation-service проверяет пользователя в identity-service;

reservation-service проверяет ресторан и столик в restaurant-catalog-service;

menu-service проверяет ресторан перед созданием меню;

review-service проверяет право на отзыв через reservation-service.

Внутренние endpoints доступны под /internal/v1 и защищены X-Service-Token.

Подготовка к брокеру сообщений

RabbitMQ/Kafka в ЛР2 не подключались, потому что это отдельный следующий этап.

При этом в коде уже есть заготовка:

общий тип DomainEvent;

интерфейс EventPublisher;

таблица outbox_events;

события reservation.status_changed и review.verified.

Сейчас события сохраняются в outbox и логируются через ConsoleEventPublisher. В следующей работе эту реализацию можно заменить на отправку в RabbitMQ/Kafka.

ORM

Работа с данными реализована через TypeORM repositories/entities. В бизнес-логике не используются прямые SQL-запросы. Каждая база имеет собственный DataSource и собственный набор entities.

Внутри одной сервисной БД связи описаны средствами ORM: например, Menu -> MenuItem -> PhotoMenuItem, Restaurant -> RestaurantCuisine, Restaurant -> RestaurantTable, Review -> PhotoReview. Между сервисами FK не создаются, потому что это нарушило бы database-per-service.

Для локального учебного запуска в .env.example включен

DATABASE_SYNCHRONIZE=true. В коде значение по умолчанию выключено, а для production режим synchronize запрещается. В промышленной версии вместо synchronize должны использоваться миграции.

Локальный запуск

```
cd labs/lab2/app
```

```
cp .env.example .env
```

```
docker compose up -d postgres
```

```
npm install
```

```
npm run build
```

```
npm run seed
```

Далее сервисы запускаются отдельными командами:

```
npm run dev:identity
```

```
npm run dev:catalog
```

```
npm run dev:menu
```

```
npm run dev:reservation
```

```
npm run dev:review
```

```
npm run dev:gateway
```

Внешний API доступен по адресу:

<http://localhost:3105/api/v1>

Вывод

В ходе работы монолитное приложение из ЛР1 было разделено на набор локальных микросервисов. Сохранен внешний API через Gateway, реализовано разделение данных по database-per-service, добавлены внутренние REST-контракты и подготовлен outbox-слой для последующего подключения брокера сообщений.